Student Final Project Report

Machine Learning Course

Course / Section:	DGE-301 Sect.1
Instructor:	Ziaullah Momand
Submission Date:	6/3/2026
Group Name:	Thapanant Khongsattra 2420110302 Pol Supmonchai 2420110187

1) Title of the Project

Project Title: Product Recommendation System for Shopping Platform Using TF-IDF and SBERT

2) Group Members Information

Full Name	Student ID	Email	Department	Role / Contribution
Thapanant Khongsattra	2420110302	2420110302@tni.ac.th	DGE	Team lead, Data collection, Preprocessing, Modeling (TF-IDF), Website development, Demo (in presentation), README writing, repository management

Pol Supmonchai	2420110187	2420110187@tni.ac.th	DGE	EDA, Feature Engineering, Modeling (SBERT),Model evaluation, Website improvement, Project refinement, Present (in presentation), Report writing

3) Department

Department: Digital Engineering

Faculty/School: Thai-Nichi International College

University : Thai-Nichi Institute of Technology

4) Background / Introduction

Explain the problem, why it matters, and what you aim to achieve.

Problem Statement

The goal of this project is to develop a **content-based recommendation system** that predicts and ranks the most relevant products for a user based on their search queries. Specifically, the system classifies a vast catalog of retail items as "relevant" or "irrelevant" to a user's intent by analyzing the semantic similarity between product features (category and name) and the user’s input.

Motivation

The recommendation system can be seamlessly integrated with an e-commerce platform to assist users in discovering their desired products more efficiently. In the modern e-commerce landscape, "information overload" represents a significant hurdle, as users are frequently overwhelmed by thousands of available products. A robust recommendation engine is essential for:

- **Improving User Experience**: Helping customers find exactly what they need quickly, thereby reducing search friction.
- **Increasing Conversion Rates**: Guiding users toward products they are likely to purchase, which directly impacts business revenue.
- **Enhancing Product Discovery**: Surfacing relevant items that the user might not have otherwise encountered, increasing the "long-tail" sales of a catalog.

Objectives

1. **Develop a Dual-Model Architecture**: Implement two distinct recommendation engines—a statistical TF-IDF model for precise keyword matching and a deep-learning Sentence Transformer (SBERT) model for semantic understanding.

2. **Optimize Text Preprocessing for Specific Models**: Apply Porter Stemming to the TF-IDF features to improve keyword flexibility while maintaining raw natural language for the SBERT model to preserve contextual meaning.

3. **Conduct a Comparative Performance Analysis**: Evaluate both models using the Business Dashboard to compare Precision@5 and Mean Reciprocal Rank (MRR), determining which architecture provides the most relevant "top-of-page" results.

Dataset Overview

- **Source**: The project utilizes a "Actual transactions from UK retailer" from Kaggle (https://www.kaggle.com/datasets/carrie1/ecommerce-data). The sliced dataset was used in this project.

Original Dataset

```
==================================================
             DATASET OVERVIEW
==================================================
Source: Kaggle (Online Retail Dataset)
Dataset Shape: 541909 rows, 8 columns

FEATURE TYPES:
InvoiceNo       object
StockCode       object
Description     object
Quantity         int64
InvoiceDate     object
UnitPrice      float64
CustomerID     float64
Country         object
dtype: object
--------------------------------------------------

MISSING VALUES:
             Total Missing  Percentage (%)
Description           1454            0.27
CustomerID          135080           24.93
--------------------------------------------------

TARGET VARIABLE:
Type: Dynamic (Similarity Score)
Definition: Cosine Similarity between Product and Query vectors.
--------------------------------------------------
DATA (First 5 Rows):
  InvoiceNo StockCode                          Description  Quantity     InvoiceDate  UnitPrice  CustomerID         Country
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6  12/1/2010 8:26       2.55     17850.0  United Kingdom
1    536365     71053                  WHITE METAL LANTERN         6  12/1/2010 8:26       3.39     17850.0  United Kingdom
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8  12/1/2010 8:26       2.75     17850.0  United Kingdom
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6  12/1/2010 8:26       3.39     17850.0  United Kingdom
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6  12/1/2010 8:26       3.39     17850.0  United Kingdom
==================================================
PS C:\Users\Paul\OneDrive\Documents\GitHub\OnlineShoppingPlatform-ML>
```

Sliced Dataset

```
==================================================
             DATASET OVERVIEW
==================================================
Source: Kaggle (Online Retail Dataset)
Dataset Shape: 4038 rows, 5 columns

FEATURE TYPES:
product_id        int64
product_name     object
category         object
price           float64
rating          float64
dtype: object
--------------------------------------------------

MISSING VALUES:
Empty DataFrame
Columns: [Total Missing, Percentage (%)]
Index: []
--------------------------------------------------

TARGET VARIABLE:
Type: Dynamic (Similarity Score)
Definition: Cosine Similarity between Product and Query vectors.
--------------------------------------------------
DATA (First 5 Rows):
   product_id                        product_name    category  price  rating
0           1   WHITE HANGING HEART T-LIGHT HOLDER  Home Decor   2.55     4.3
1           2                  WHITE METAL LANTERN      Office   3.39     3.7
2           3       CREAM CUPID HEARTS COAT HANGER  Home Decor   2.75     3.1
3           4  KNITTED UNION FLAG HOT WATER BOTTLE     Kitchen   3.39     3.2
4           5       RED WOOLLY HOTTIE WHITE HEART.       Gifts   3.39     3.3
==================================================
PS C:\Users\Paul\OneDrive\Documents\GitHub\OnlineShoppingPlatform-ML>
```

Scope :

While this project was initially conceived as a comprehensive Online Shopping Platform, the scope was refined to focus specifically on the Recommendation System. This allows for studying deeper technical applications of Machine Learning.

- **What Was Accomplished (In-Scope) :**
    - Data Engineering: Systematic loading, cleaning, and preprocessing of the Dataset.
    - Exploratory Data Analysis (EDA): Conducted in a separate environment to identify data distributions, missing values, and product variety.
    - Hybrid Modeling: Development of a recommendation engine using both TF-IDF (keyword matching) and SBERT (semantic meaning).
    - System Evaluation: Quantitative testing using metrics such as Recall, Precision @ 5, and Mean Reciprocal Rank (MRR)
    - Administrative Visualisations: Created a dedicated Admin Dashboard using streamlit metrics and charts

- **What Was Not Accomplished (Out-of-Scope) :**
    - Real-time Trend Analysis: While the dataset contains temporal data, true trend analysis requires live, streaming data to be statistically significant.
    - Full E-commerce Functionality: Features such as user authentication, shopping carts, and payment gateways were excluded to prevent the "web development" aspect from overshadowing the "machine learning" objectives.

5) Literature Review

Reference 1: "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" (Reimers & Gurevych, 2019)

- **Problem:** Standard BERT and RoBERTa architectures were computationally prohibitive for large-scale semantic similarity searches in real-time.
- **Method/Model:** Introduced SBERT, which uses Siamese and triplet network structures to derive semantically meaningful sentence embeddings.
- **Key Result:** Reduced the time for finding the most similar pair from 65 hours with BERT to about 5 seconds with SBERT, enabling the real-time discovery engine used in our project.
- **Link:** https://arxiv.org/abs/1908.10084

Reference 2: "Feature Extraction: TF-IDF Vectorization" (Scikit-learn Documentation / Pedregosa et al., 2011)

- **Problem:** Machine Learning models cannot process raw strings; they require numerical input that reflects not just word frequency, but the relative importance of a word within a specific product catalog.
- **Method/Model:** The TF-IDF algorithm, which rescales the frequency of words by how often they appear across all descriptions, effectively filtering out common "stop words" and highlighting unique product keywords.
- **Key Result:** Provides a robust "Lexical" baseline for the project, allowing the system to perform high-speed, exact keyword matches for specific item searches.
- **Link:** https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

Reference 3: "E-Commerce Products Recommendation" (Youtube / AI with Noor, 2024)

- **Problem:** -
- **Method/Model:** Developed a Full-Stack E-commerce Website utilizing the Flask Framework for the backend and TF-IDF as the primary search and recommendation model.
- **Key Result:** Produced a comprehensive platform capable of product discovery through keyword search, as well as integrated user management features such as Login/Registration and Password Creation.
- **Link:** https://www.youtube.com/watch?v=30LmKbBSpAs&list=PLdF3rLdF4ICTVTjSpdX5GCrHBGedaXoru

Reference 4: "Streamlit: The Fastest Way To Build Python Apps?" (Youtube / pixegami, 2023)

- **Problem:** -
- **Method/Model:** A Basic Streamlit Framework tutorial focusing on building interactive data scripts and deploying them as web apps using Python-only syntax.
- **Key Result:** Demonstrated how to use core components like sidebars, file uploaders, and data tables to turn static code into a functional, interactive user interface.
- **Link:** https://youtu.be/D0D4Pa22iG0?si=l5zCt2j1jA62it70

Gap and Difference:

- **Dataset and Preprocessing :** This project utilizes the UCI Online Retail Dataset which is different from references. Although the foundational preprocessing steps—such as handling missing values and removing duplicates—are similar to the methodology(in Reference 3).
- **Modeling and Evaluation:** The primary difference in modeling lies in the Hybrid Approach adopted for this project. Whereas the referenced works typically implement a single model—such as the TF-IDF search used in Reference 2 or the semantic embeddings explored in Reference 1—this project combines both TF-IDF and SBERT into a unified system. This allows for a direct comparison between lexical keyword matching and deep semantic discovery. This project incorporates a rigorous scientific evaluation using Recall, Mean Reciprocal Rank (MRR) and Precision @ 5 to mathematically validate the performance of this dual-model architecture.

6) Method of Work

Describe your full pipeline clearly so someone else can reproduce your approach.

Data Collection & Source

- **Source:** The dataset used in this project was obtained from Kaggle. (https://www.kaggle.com/datasets/carrie1/ecommerce-data)
- **Suitability:** This is a transactional dataset containing actual purchases from a UK-based retailer. It includes customer, product, and transaction details, which provides the necessary context to use in this recommendation system project.

Preprocessing

- **Missing values:** In the dataset were handled during the data loading stage. If the purchase_date field was missing, the system attempted to use the InvoiceDate column. If neither column existed, a default date was assigned to ensure consistency and prevent errors during processing.

```
# Handle missing 'purchase_date'
if 'purchase_date' not in transactions.columns:
    # Check for Kaggle's 'InvoiceDate'
    if 'InvoiceDate' in transactions.columns:
        transactions = transactions.rename(columns={'InvoiceDate': 'purchase_date'})
    else:
        transactions['purchase_date'] = pd.to_datetime("2024-01-01")

transactions['purchase_date'] = pd.to_datetime(transactions['purchase_date'])
return products, transactions
```

- **Duplicates:** The dataset was checked for duplicate records.. The result indicated that there were no duplicate rows in the dataset, so no duplicate removal was required.

```
print("Products Duplcates :", products.duplicated().sum())
print("Transactions Duplcates :", products.duplicated().sum())
```

```
Products Duplcates : 0
Transactions Duplcates : 0
```

Outliers: There were no significant outliers that would negatively affect the dataset in the first place. Outliers in product prices were managed through a budget filtering mechanism. Users can specify a maximum price range, which prevents extremely expensive products from appearing in the recommendation results.

```
with col_input3:
    max_price = st.slider("Max Budget ($)", 0, 500, 100)
```

```
# Apply Filters
mask = df_products['price'] <= max_price
```

Encoding: Since machine learning models require numerical input, the textual features in the dataset were encoded into numerical representations. Product name and category information were converted into vector representations using TF-IDF and SBERT embeddings. These encoded vectors were later used by the recommendation models to calculate similarity between products.

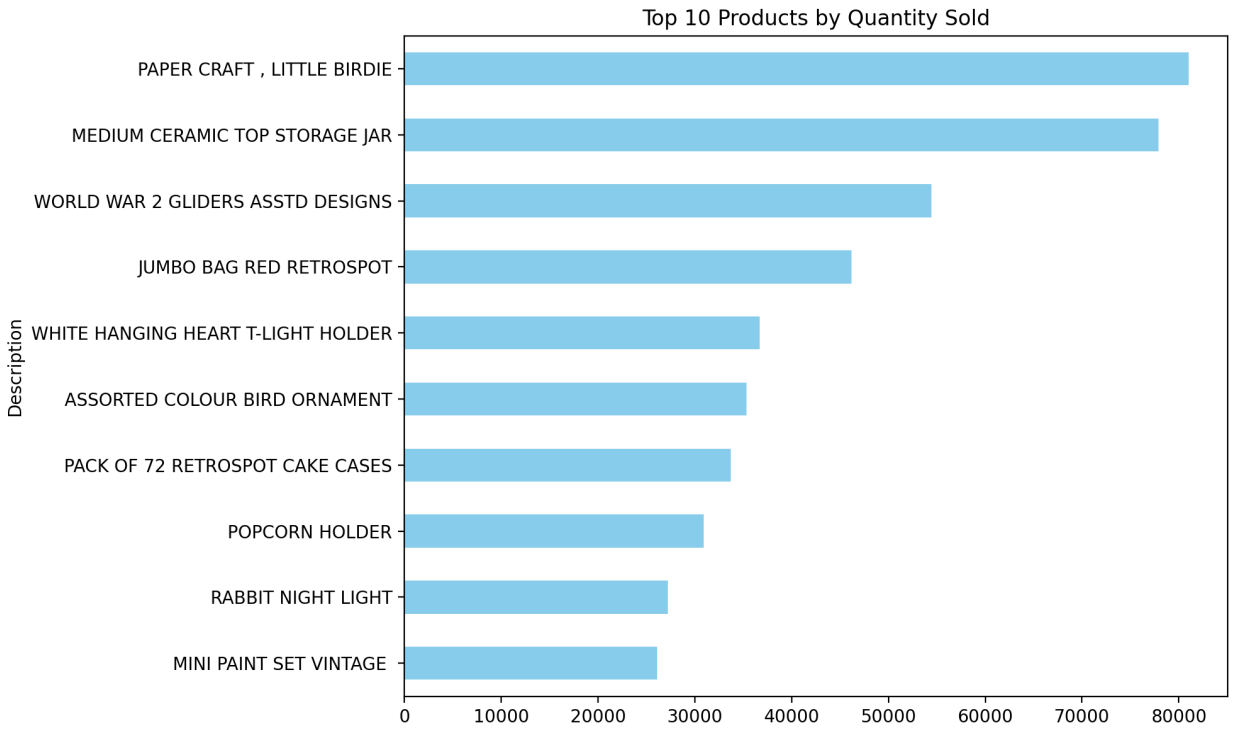
Scaling: Feature scaling was not required in this project because TF-IDF and SBERT embeddings inherently produce normalized vector representations suitable for similarity computation.

Split Ratios: Instead of a traditional train-test split, the recommender system was evaluated using random sampling of product items. A subset of up to 50 products was selected to measure recommendation quality using Recall, Precision, and Mean Reciprocal Rank (MRR).

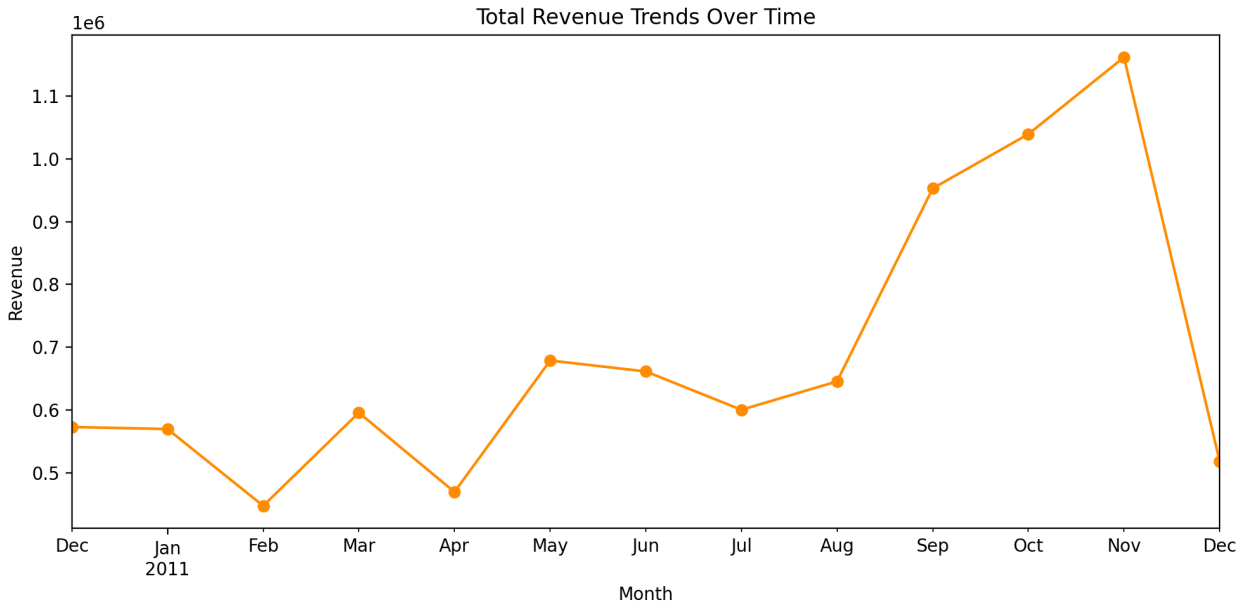
Exploratory Data Analysis (EDA)

While the core of this project focuses on recommendation algorithms, the following Exploratory Data Analysis (EDA) was performed to establish a business baseline for the platform. These visualizations do not directly influence the similarity calculations; rather, they allow us to observe the platform's performance, identify high-value customer segments, and understand the temporal patterns of demand—all of which are vital for interpreting the final model’s impact in a real-world production environment.

Key insight of the platform of quantity sold from the platform



Key insight of the total revenue trends over time



Feature Engineering

In this project text preprocessing was applied by combining the product category and product name into a single feature. Porter stemming was then used to reduce words to their root form, which improves the efficiency of text-based similarity models.

```
# 1. Feature Engineering
df['features_clean'] = df['category'].astype(str) + " " + df['product_name'].astype(str)
ps = PorterStemmer()
df['features_stemmed'] = df['features_clean'].apply(lambda x: " ".join([ps.stem(word) for word in x.split()]))
```

Modeling

1.TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF is a text vectorization technique that converts textual data into numerical vectors based on the frequency and importance of words within the dataset. In this project, TF-IDF was applied to the processed product features to represent each product as a numerical vector. Cosine similarity was then used to measure the similarity between product vectors and generate recommendations.

This method focuses on keyword matching, meaning that products with similar words in their descriptions or categories will have higher similarity scores.

```
# 2. TF-IDF Setup
tfidf = TfidfVectorizer(stop_words='english', ngram_range=(1, 2))
tfidf_matrix = tfidf.fit_transform(df['features_stemmed'])
tfidf_sim = cosine_similarity(tfidf_matrix)
```

2. SBERT (Sentence-BERT)

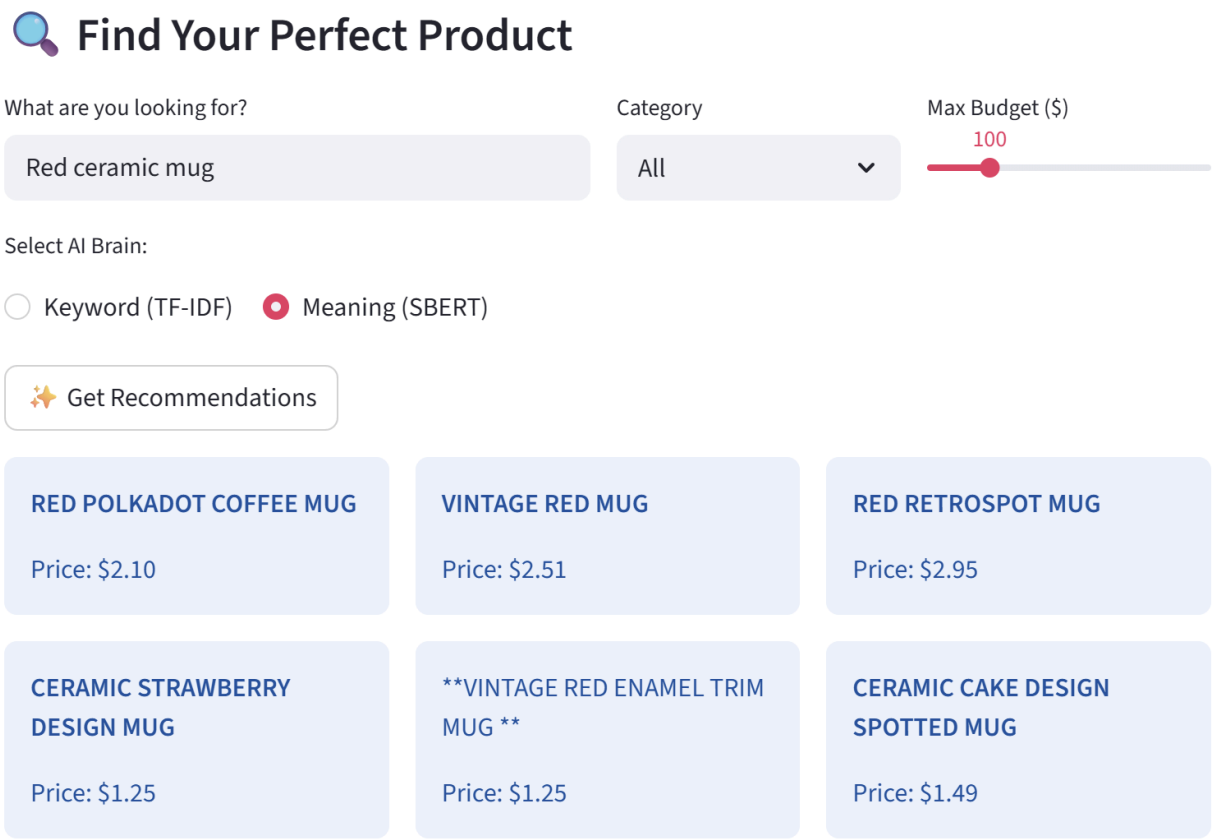
Sentence-BERT is a deep learning model that generates semantic embeddings for text. Unlike TF-IDF, SBERT captures the contextual meaning of sentences, allowing the model to understand relationships between words and phrases more effectively.

In this project, SBERT was used to encode product features into dense vector embeddings. Cosine similarity was then applied to measure the semantic similarity between products and recommend items that are conceptually related.

```
# 3. SBERT Setup
sbert_model = SentenceTransformer('all-MiniLM-L6-v2')
sbert_embeddings = sbert_model.encode(df['features_clean'].tolist(), show_progress_bar=False)
sbert_sim = cosine_similarity(sbert_embeddings)

return tfidf, tfidf_matrix, tfidf_sim, sbert_model, sbert_sim, sbert_embeddings

df, tfidf_matrix, tfidf_sim, sbert_model, sbert_sim, sbert_embeddings = init_models(df_products)
```



Hyperparameters

Several hyperparameters were configured to improve the performance of the recommendation models.

For the TF-IDF model, the following parameters were used:

- **stop_words = 'english'**
  Removes common English words that do not contribute meaningful information.

- **ngram_range = (1,2)**
  Allows the model to consider both single words (unigrams) and pairs of words (bigrams).

For the SBERT model, the pre-trained model all-MiniLM-L6-v2 was used. This model is widely used for semantic similarity tasks because it provides a good balance between performance and computational efficiency.

Training Process

Since this project implements a content-based recommendation system, the models do not require traditional supervised training with labeled data. Instead, the training process involves generating vector representations of product features.

First, the textual features were created by combining the product category and product name. These features were then preprocessed using stemming to normalize the words.

Next, the TF-IDF vectorizer was fitted to the processed text to generate a TF-IDF matrix representing each product. For the SBERT approach, the pre-trained model encoded the product features into semantic embeddings.

After the vector representations were generated, cosine similarity was calculated between all product vectors to determine the similarity scores. Products with the highest similarity scores were selected as recommendations.

Evaluation

Evaluation Metrics

To evaluate the performance of the recommendation system, several metrics were used, including Precision, Recall, and Mean Reciprocal Rank (MRR). These metrics help measure how effectively the system recommends relevant products to users.

Precision

Precision measures the proportion of recommended items that are actually relevant. In this project, Precision@5 was used, meaning that the metric evaluates how many of the top five recommended products are relevant to the user query. Precision is appropriate for this task because recommendation systems aim to present users with a small number of highly relevant products.

Recall

Recall measures the ability of the model to identify all relevant items from the dataset. It calculates the proportion of relevant products that were successfully recommended by the system. Recall is useful for understanding how well the system captures all potentially relevant products for a user query.

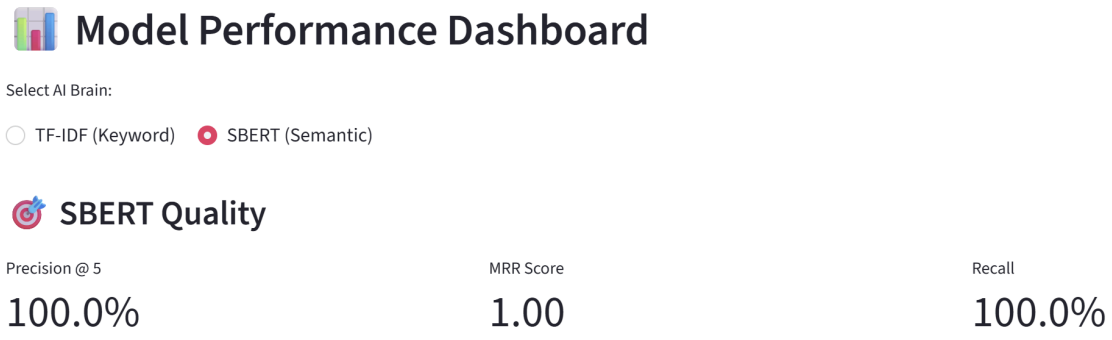
Mean Reciprocal Rank (MRR)

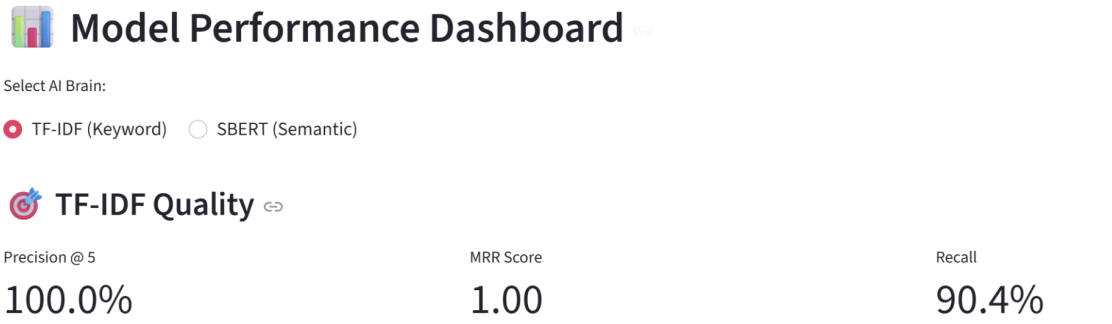
Mean Reciprocal Rank (MRR) measures how quickly the system returns a relevant recommendation. It calculates the average reciprocal rank of the first relevant item in the recommendation list. MRR is particularly suitable for recommendation systems because users typically interact with only the top few results. A higher MRR indicates that relevant products appear earlier in the recommendation list.

7) Results

Results comparison table:

Model	Key Settings	Precision@5	MRR Score	Recall	Notes
TF-IDF	TF-IDF Vectorizer, stopwords removed, n-gram (1,2), cosine similarity	100%	1.00	90.4%	Performs well for keyword-based similarity but may miss semantic meaning between products
SBERT	SentenceTransformer (all-MiniLM-L6-v2), semantic embeddings, cosine similarity	100%	1.00	100.0%	Captures semantic meaning of product descriptions and achieves slightly better recall





Note on Evaluation Metrics

It should be noted that this project does not utilize a confusion matrix or regression error analysis metrics.

In this project, the system is designed as a content-based recommendation system that ranks products based on similarity rather than predicting class labels or numerical values. Therefore, evaluation metrics such as Precision@K, Recall@K, and Mean Reciprocal Rank (MRR) are more appropriate for measuring the quality of the ranked recommendations produced by the models.

Model Selection and Justification

Both TF-IDF and SBERT models were evaluated to determine which algorithm performs best for the recommendation system. The comparison was conducted using Precision@5, Recall, and Mean Reciprocal Rank (MRR), which are appropriate metrics for evaluating ranking-based recommendation systems.

The results show that SBERT achieved the highest overall performance, with 100% Precision@5, 100% Recall, and an MRR score of 1.00, slightly outperforming the TF-IDF model in terms of recall. While TF-IDF also achieved perfect precision and MRR, its recall score (90.4%) indicates that some relevant items were not successfully retrieved.

The reason SBERT performs better is that it uses semantic embeddings, allowing the model to understand the meaning and context of product descriptions, rather than relying only on keyword matching. In contrast, TF-IDF focuses mainly on word frequency and textual similarity, which may miss recommendations when similar products use different wording.

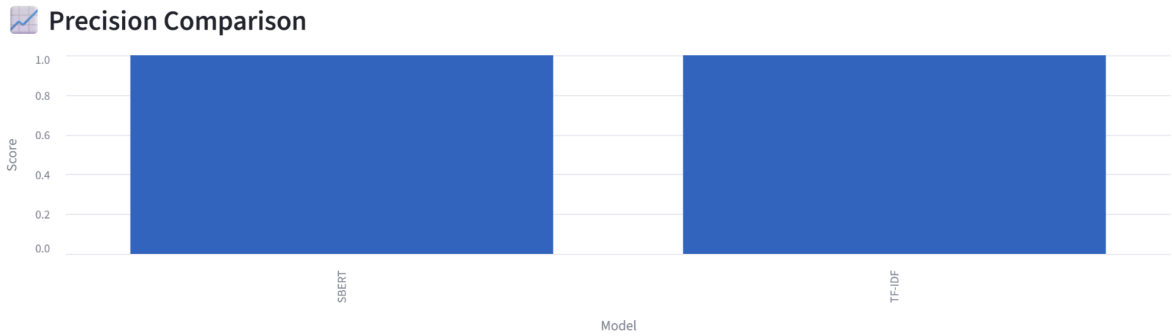
Therefore, SBERT was selected as the final model because it provides better semantic understanding, higher recall, and more reliable recommendation coverage, making it more suitable for a product recommendation system.

Failure Cases and Common Errors

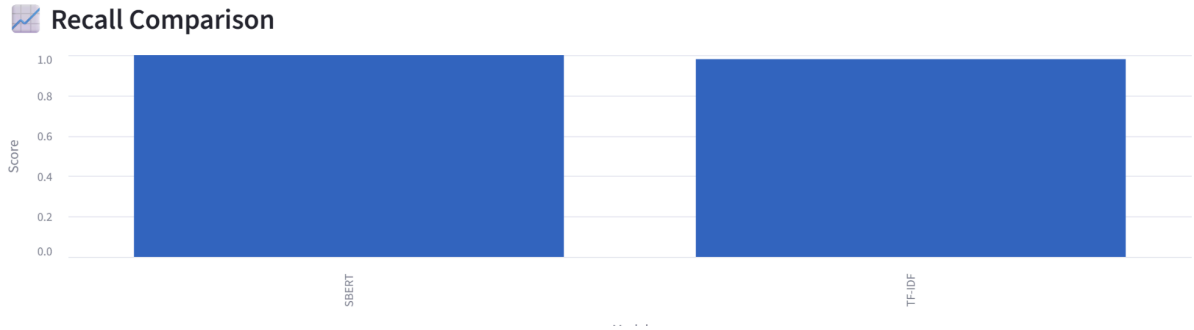
Some failure cases may occur in this recommendation system. The TF-IDF model may miss relevant products when similar items use different wording, since it relies on keyword matching. Although SBERT improves semantic understanding, it may still generate incorrect recommendations when product descriptions are too short or ambiguous. Additionally, the system only uses product text features, so the recommendation quality may be limited without user behavior data such as purchase history or ratings.

Key plots / figures included:

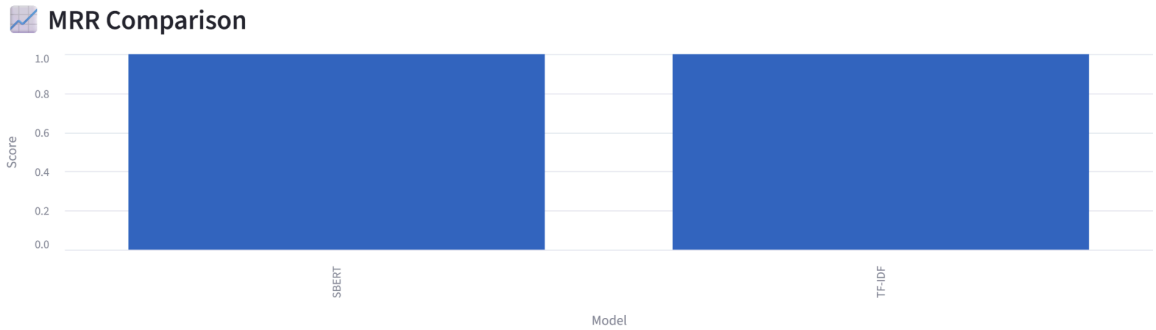
1. **Figure 1:**



2. **Figure 2:**



3. **Figure 3:**



8) Conclusion

Key Findings

- Both models achieved 100% Precision@5, indicating that most recommended items were relevant to the query.
- The MRR score reached 1.00 for both models, meaning relevant items were usually ranked at the top of the recommendation list.
- SBERT achieved a higher Recall (100%) compared to TF-IDF (90.4%), suggesting that SBERT can capture deeper semantic relationships between product descriptions.
- The semantic embedding approach (SBERT) performs better when user queries use different wording than the product names.
- TF-IDF works effectively for keyword-based search, but may struggle when the query meaning differs from the exact words used in product descriptions.

Best Model and Final Performance

Among the evaluated models, **SBERT was selected as the best-performing model** due to its stronger ability to capture semantic meaning in text.

Final performance results:

Model	Precision@5	MRR Score	Recall
TF-IDF	100%	1.00	90.4%
SBERT	100%	1.00	100.0%

Limitations

Several limitations should be acknowledged in this project.

- **Incomplete initial scope:** The project scope was narrowed to focus primarily on the recommendation system, therefore full e-commerce functionalities such as user authentication, shopping carts, and payment gateways were not implemented.
- **No real-time processing:** Although the dataset contains purchase date information, real-time trend analysis was not performed because meaningful trend detection typically requires live or continuously updated data rather than a static dataset.
- **Unsatisfied evaluation methods:** Some evaluation methods such as confusion matrix or regression error analysis are not applicable because the system is a ranking-based recommender system rather than a classification or regression model.
- **Sliced Dataset:** The dataset used in this project was relatively small from the sliced approach, which may lead to overly high evaluation scores. This approach may not fully reflect real-world performance, and using a larger dataset could improve the reliability and robustness of the model.

Future Work

- **Code Quality Improvement:** One potential improvement is enhancing the cleanliness and organization of the code. While the main focus was on implementing and testing the recommendation models, some parts of the code could be refactored to improve readability and structure. Improving code formatting, naming, and modularity would make the system easier to maintain and extend in the future.
- **Use a Larger and More Diverse Dataset:** The recommendation system could be improved by using larger and more diverse dataset. A bigger dataset with more products and user interactions would allow the model to learn richer relationships between items and potentially improve recommendation quality.
- **Incorporate User Behavior Data:** Future versions of the system probably could incorporate additional user behavior information such as purchase history, browsing patterns, or product ratings. This would allow the recommendation system to move toward a personalized recommendation approach rather than relying only on product similarity.

9) Project Source Code Availability Link

Repository Link: https://github.com/DokkDokki/OnlineShoppingPlatform-ML

How to run (brief):

1. Clone the Repository
2. Create the virtual environment
3. Install required Libraries
4. Download the Dataset
5. Rename the file to data.csv.
6. Create a folder named data inside the project directory. Move data.csv into that folder.
7. Run the Converter: We use a script to clean the data and simulate "Categories" and "Ratings" (since the raw dataset lacks them).
8. Run “streamlit run app.py”

Dataset link/location: https://www.kaggle.com/datasets/carrie1/ecommerce-data